Advanced Sorting Algorithms Summary

1. Non-Comparison & Counting-Based Sorting

Algorithm	Definition	Advantages	Disadvantages	Time Complexity	Space Complexity
Counting Sort	Non-comparison algorithm using key counts to determine element positions.	• Linear time O(n + k) for small ranges • Stable sorting method • Efficient for discrete integers	• Requires O(k) memory array • Inefficient for large ranges (k >> n) • Strictly restricted to integer/discrete keys	**Best/Avg/Worst:** O(n + k)	O(n + k)
Radix Sort	Stable algorithm that processes digits sequentially (LSD/MSD) via stable sub-sort.	• Linear-time potential for fixed-length keys • Preserves order (stable) • Highly efficient for integers/strings	• Memory-intensive O(n + r) • Dependent on key length/radix • Poor performance on variable-length keys	**Best/Avg/Worst:** O(k(n + r))	O(n + r)
Bucket Sort	Distributes elements into ranges (buckets), sorts individual buckets, and concatenates.	• Near-linear average performance • Excellent for uniformly distributed data • Independent bucket processing	• Degrades to $O(n^2)$ if data clusters • Extra allocation overhead for buckets • Highly sensitive to input distribution	**Best/Avg:** O(n + k) **Worst:** $O(n^2)$	O(n + k)

2. Distributed & Parallel Sorting Systems

Algorithm	Definition	Advantages	Disadvantages	Time Complexity	Space Complexity
Parallel Sorting (General)	Task-decomposed model distributing sorting workloads concurrently across multi-core processors.	• Minimizes overall execution time • Maximizes CPU core utilization • Scalable for high-volume datasets	• Thread management overhead • Synchronization & lock contention • Performance degrades on over-subscription	**Task Dependent:** Ideally $O(T_{seq} / p)$	Algorithm Dependent
Parallel Quick Sort	Concurrent partitioning of sub-arrays executed across parallel threads.	• High speedup over sequential Quick Sort • Independent sub-array execution • Excellent CPU utilization	• Vulnerable to partition imbalance • High thread context-switching costs • Deep call-stack memory overhead	**Best/Ideal:** $O(n / p)$ **Avg:** $O(n \log n / p)$ **Worst:** $O(n^2)$	**Best/Avg:** $O(\log n)$ **Worst:** $O(n)$
Parallel Merge Sort	Parallel division of data arrays into sub-problems, followed by synchronized concurrent merges.	• Predictable $O(n \log n)$ baseline performance • Natural task-decomposition structure • High throughput on large datasets	• High $O(n)$ memory overhead for merge step • Merge synchronization barriers • Inter-core communication overhead	**Avg/Worst:** $O(n \log n / p)$	$O(n)$

3. Sorting Networks & Hardware Algorithms

Algorithm	Definition	Advantages	Disadvantages	Time Complexity	Space Complexity
Odd-Even Transposition Sort	Parallel comparison-sort alternating odd/even index-pair adjacent swaps across phases.	• Simple SIMD/parallel structure • Simultaneous comparisons per phase • Directly maps to hardware grids	• Poor efficiency (O(n) parallel phases) • Sub-optimal scaling for large n • High inter-phase synchronization barrier	**Parallel:** O(n) (with p = n processors)	O(n)
Bitonic Merge Sort	Sorting network model building bitonic sequences to enable divide-and-conquer parallel merges.	• Fully deterministic execution path • Ideal for parallel/GPU hardware mapping • Fixed comparison-exchange layout	• Higher total operations than standard merge • Constrained network topology (n = 2^k) • Higher architectural complexity	**Parallel:** $O(\log^2 n)$ (with p = n processors)	O(n)
Odd-Even Merge Sort	Sorting network that recursively splits and merges odd and even indexed sub-sequences.	• Highly regular parallel structure • Efficient hardware/circuit implementation • Predictable execution bounds	• Complex recursive routing network • Inefficient for small input sizes • Additional buffer memory overhead	**Parallel:** $O(\log^2 n)$ (with p = n processors)	O(n)

4. Disk-Bound & External Systems

Algorithm	Definition	Advantages	Disadvantages	Time Complexity	Space Complexity
External Sorting	Multi-phase sorting (run generation & multi-way merge) for datasets exceeding RAM limits.	● Handles arbitrarily large datasets (> RAM) ● Efficient utilization of disk/block I/O ● Scalable for enterprise Big Data	● Heavily bottlenecked by Disk I/O speed ● High read/write storage overhead ● Substantial bus transfer latency	**I/O Bound:** O(n log n) compute work; Performance dominated by I/O cycles.	O(n) external storage